

1 Computational Geometry

1.1 convex-hull

```

1 struct node{
2     int x, y;
3     node operator - (const node &b) {
4         return {x - b.x, y - b.y};
5     }
6     int operator * (const node &b) {
7         return x * b.y - y * b.x;
8     }
9     bool operator < (const node &b) {
10        if (x != b.x) return x < b.x;
11        return y < b.y;
12    }
13 };
14 vector<node> build(vector<node> v)
15 {
16     vector<node> ans;
17     sort(v.begin(), v.end());
18     for(int i = 0; i < v.size(); i++){
19         while(ans.size() > 1 and (ans.end()
20             [-2] - ans.end()[-1]) * (ans.end()
21             )[-2] - v[i]) > 0){
22             ans.pop_back();
23         }
24         ans.pb(v[i]);
25     }
26     for(int i = v.size() - 2; i >= 0; i--){
27         while(ans.size() > 1 and (ans.end()
28             [-2] - ans.end()[-1]) * (ans.end()
29             )[-2] - v[i]) > 0){
30             ans.pop_back();
31         }
32         ans.pb(v[i]);
33     }
34     ans.pop_back();
35     return ans;
36 }

```

1.2 最近點對

```

1 set<pii> s; // {y, x}
2 for(int i = 0; i < n; i++){
3     int d = sqrt(ans) + 1;
4     while(s.size() > 0 and arr[i].F - arr[id]
5         ].F >= d){
6         s.erase({arr[id].S, arr[id].F});
7         id++;
8     }
9     auto l = s.lower_bound({arr[i].S - d, -1
10         e9});
11     auto r = s.lower_bound({arr[i].S + d, 1
12         e9});
13     for(auto j = l; j != r; ++j) ans = min(
14         ans, dis(i, *j));
15     s.insert({arr[i].S, arr[i].F});
16 }

```

1.3 李超線段樹

```

1 struct Line{
2     int k, b;
3     int val(int x){
4         return k * x + b;
5     }
6 };
7 Line tree[(int)1e6];
8 bool has_line[(int)1e6];
9 void insert(int node, int l, int r, Line
10     new_line)
11 {
12     if(!has_line[node]){
13         tree[node] = new_line;
14         has_line[node] = true;
15         return;
16     }
17     int mid = (l + r) >> 1;
18     bool l_better = new_line.val(l) > tree[
19         node].val(l);
20     bool mid_better = new_line.val(mid) >
21         tree[node].val(mid);
22     if(mid_better) swap(new_line, tree[node]
23         );
24     if(l == r) return;
25     if(l_better != mid_better) insert(node
26         << 1, l, mid, new_line);
27     else insert(node << 1 | 1, mid + 1, r,
28         new_line);
29 }
30 int query(int node, int l, int r, int x)
31 {
32     if(!has_line[node]) return -2e18;
33     int re = tree[node].val(x);
34     if(l == r) return re;
35     int mid = (l + r) >> 1;
36     if(x <= mid) re = max(re, query(node <<
37         1, l, mid, x));
38     else re = max(re, query(node << 1 | 1,
39         mid + 1, r, x));
40     return re;
41 }

```

2 Data Structure

2.1 persistent-segment-tree

```

1 #include <bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<long long, long long>
7 #define F first
8 #define S second
9 #define int long long
10 struct st{
11     int l, r, sum;
12 }tree[(int)8e6 + 5];

```

```

12 int root[(int)2e5 + 5], ar[(int)2e5 + 5];
13 int copy_cnt = 1, cnt = 0;
14 int build(int l, int r)
15 {
16     int node = ++cnt;
17     if(l == r){
18         tree[cnt].sum = ar[l - 1];
19         return node;
20     }
21     int mid = (l + r) >> 1;
22     tree[node].l = build(l, mid);
23     tree[node].r = build(mid + 1, r);
24     tree[node].sum = tree[tree[node].l].sum
25         + tree[tree[node].r].sum;
26     return node;
27 }
28 int upd(int pre, int l, int r, int pos, int
29     val)
30 {
31     int node = ++cnt;
32     tree[node] = tree[pre];
33     if(l == r){
34         tree[node].sum = val;
35         return node;
36     }
37     int mid = (l + r) >> 1;
38     if(pos <= mid) tree[node].l = upd(tree[
39         pre].l, l, mid, pos, val);
40     else tree[node].r = upd(tree[pre].r, mid
41         + 1, r, pos, val);
42     tree[node].sum = tree[tree[node].l].sum
43         + tree[tree[node].r].sum;
44     return node;
45 }
46 int query(int node, int l, int r, int ql,
47     int qr)
48 {
49     if(ql <= l and r <= qr) return tree[node
50         ].sum;
51     int mid = (l + r) >> 1, re = 0;
52     if(ql <= mid) re += query(tree[node].l,
53         l, mid, ql, qr);
54     if(qr > mid) re += query(tree[node].r,
55         mid + 1, r, ql, qr);
56     return re;
57 }
58 signed main()
59 {
60     KCC
61     int n, q;
62     cin >> n >> q;
63     for(int i = 0; i < n; i++) cin >> ar[i];
64     root[copy_cnt] = build(1, n);
65     while(q--){
66         int op, k;
67         cin >> op >> k;
68         if(op == 1){
69             int a, x;
70             cin >> a >> x;
71             root[k] = upd(root[k], 1, n, a,
72                 x);
73         }
74         if(op == 2){
75             int l, r;
76             cin >> l >> r;

```

```

67         cout << query(root[k], 1, n, l,
68             r) << "\n";
69     }
70     if(op == 3) root[++copy_cnt] = root[
71         k];
72     return 0;
73 }

```

2.2 segment tree lazytag

```

1 struct SegmentTree{
2     struct Node{
3         int sum, f, d, len;
4     } tree[MAXN << 2];
5     void pull(int id){
6         tree[id].sum = tree[id << 1].sum +
7             tree[id << 1 | 1].sum;
8     }
9     void addtag(int id, int a, int d){
10        tree[id].f += a;
11        tree[id].d += d;
12        tree[id].sum += (2 * a + (tree[id].
13            len - 1) * d) * tree[id].len /
14            2;
15    }
16    void push(int id){
17        if(!tree[id].f && !tree[id].d)
18            return;
19        int mid_f = tree[id].f + tree[id <<
20            1].len * tree[id].d;
21        addtag(id << 1, tree[id].f, tree[id
22            ].d);
23        addtag(id << 1 | 1, mid_f, tree[id].
24            d);
25        tree[id].f = tree[id].d = 0;
26    }
27    void build(int l, int r, int id, int ar
28        []){
29        tree[id].len = r - l + 1;
30        tree[id].f = tree[id].d = 0;
31        if(l == r){
32            tree[id].sum = ar[l];
33            return;
34        }
35        int mid = (l + r) >> 1;
36        build(l, mid, id << 1, ar);
37        build(mid + 1, r, id << 1 | 1, ar);
38        pull(id);
39    }
40    void update(int ql, int qr, int a, int d
41        , int l, int r, int id){
42        if(ql <= l and r <= qr){
43            addtag(id, a + (l - ql) * d, d);
44            return;
45        }
46        push(id);
47        int mid = (l + r) >> 1;
48        if(ql <= mid) update(ql, qr, a, d, l
49            , mid, id << 1);
50        if(qr > mid) update(ql, qr, a, d,
51            mid + 1, r, id << 1 | 1);
52        pull(id);

```

```

42 }
43 int query(int ql, int qr, int l, int r,
44         int id){
45     if(ql <= l and r <= qr) return tree[
46         id].sum;
47     push(id);
48     int mid = (l + r) >> 1, re = 0;
49     if(ql <= mid) re += query(ql, qr, l,
50         mid, id << 1);
51     if(qr > mid) re += query(ql, qr, mid
52         + 1, r, id << 1 | 1);
53     return re;
54 }
55 };

```

2.3 treap

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define KCC ios::sync_with_stdio(0);cin.tie
4 (0);
5 #define pb push_back
6 #define pii pair<int, int>
7 #define F first
8 #define S second
9 mt19937 rnd(time(0));
10 const int MAXN = 2e5 + 5;
11 int left_node[MAXN], right_node[MAXN],
12 priority[MAXN], sz[MAXN], node_cnt = 0,
13 root = 0;
14 char val[MAXN];
15 void up(int i)
16 {
17     sz[i] = sz[left_node[i]] + sz[right_node
18         [i]] + 1;
19 }
20 void split(int i, int l, int r, int rank)
21 {
22     if(i == 0){
23         left_node[r] = right_node[l] = 0;
24         return;
25     }
26     if(sz[left_node[i]] + 1 <= rank){
27         right_node[l] = i;
28         split(right_node[i], i, r, rank - sz
29             [left_node[i]] - 1);
30     }
31     else{
32         left_node[r] = i;
33         split(left_node[i], l, i, rank);
34     }
35     up(i);
36 }
37 int merge(int l, int r)
38 {
39     if(l == 0 or r == 0) return l + r;
40     if(priority[l] >= priority[r]){
41         right_node[l] = merge(right_node[l],
42             r);
43         up(l);
44         return l;
45     }
46     else{
47         left_node[r] = merge(l, left_node[r
48             ]);
49         up(r);
50         return r;
51     }
52 }
53 void out(int i)
54 {
55     if(i == 0) return;
56     out(left_node[i]);
57     cout << val[i];
58     out(right_node[i]);
59 }
60 int main()
61 {
62     KCC
63     int n, m;
64     cin >> n >> m;
65     for(int i = 0; i < n; i++){
66         char c;
67         cin >> c;
68         priority[++node_cnt] = rnd();
69         val[node_cnt] = c;
70         sz[node_cnt] = 1;
71         root = merge(root, node_cnt);
72     }
73     while(m--){
74         int a, b;
75         cin >> a >> b
76         split(root, 0, 0, b);
77         int lm = right_node[0];
78         int r = left_node[0];
79         split(lm, 0, 0, a - 1);
80         int l = right_node[0];
81         int m = left_node[0];
82         root = merge(merge(l, r), m);
83     }
84     out(root);
85     return 0;
86 }

```

3 Flow

3.1 dinic

```

1 struct Dinic{
2     struct Edge{
3         int to, cap, rev;
4     };
5     vector<Edge> adj[505];
6     vector<int> level, ptr;
7     int n;
8     Dinic(int _n){
9         n = _n;
10    }
11    void addedge(int u, int v, int w){
12        adj[u].pb({v, w, adj[v].size()});
13        adj[v].pb({u, 0, adj[u].size() - 1});
14    }
15    bool bfs(int s, int t)

```

```

16 {
17     fill(level.begin(), level.end(), -1)
18     ;
19     level[s] = 0;
20     queue<int> q;
21     q.push(s);
22     while(q.size()){
23         int x = q.front(); q.pop();
24         for(auto edge : adj[x]){
25             if(edge.cap > 0 and level[
26                 edge.to] == -1){
27                 level[edge.to] = level[x
28                     ] + 1;
29                 q.push(edge.to);
30             }
31         }
32     }
33     return (level[t] != -1);
34 }
35 int dfs(int now, int t, int pushed)
36 {
37     if(now == t) return pushed;
38     for(int &i = ptr[now]; i < adj[now].
39         size(); i++){
40         auto &w = adj[now][i];
41         if(level[now] + 1 != level[w.to]
42             or w.cap == 0) continue;
43         int tr = dfs(w.to, t, min(pushed
44             , w.cap));
45         if(tr == 0) continue;
46         w.cap -= tr;
47         adj[w.to][w.rev].cap += tr;
48         return tr;
49     }
50     return 0;
51 }
52 int compute_max_flow(){
53     int max_flow = 0;
54     while(bfs(1, n)){
55         fill(ptr.begin(), ptr.end(), 0);
56         while(int pushed = dfs(1, n, 1
57             e18)){
58             max_flow += pushed;
59         }
60     }
61     return max_flow;
62 }
63 };

```

3.2 Edmonds-Karp

```

1 struct max_flow{
2     int n;
3     vector<int> dis, par, mn;
4     vector<vector<int>> g;
5     vector<array<int, 3>> edge;
6     max_flow(int _n){
7         n = _n;
8         dis.resize(n + 5);
9         par.resize(n + 5);
10        mn.resize(n + 5);
11        g.resize(n + 5);
12    }

```

```

13 void add_edge(int a, int b, int c){
14     g[a].pb(edge.size());
15     edge.pb({a, b, c});
16     g[b].pb(edge.size());
17     edge.pb({b, a, 0});
18 }
19 int bfs()
20 {
21     for(int i = 1; i <= n; i++) dis[i] =
22         -1;
23     for(int i = 1; i <= n; i++) mn[i] =
24         1e18;
25     dis[1] = 0;
26     queue<int> q;
27     q.push(1);
28     while(q.size()){
29         int x = q.front(); q.pop();
30         for(int i : g[x]){
31             auto [now, to, w] = edge[i];
32             if(dis[to] == -1 and w > 0){
33                 dis[to] = dis[now] + 1;
34                 par[to] = i;
35                 mn[to] = min(mn[now], w);
36                 q.push(to);
37             }
38         }
39     }
40     if(dis[n] == -1) return 0;
41     int now = n;
42     while(now != 1){
43         auto &[a, b, w] = edge[par[now]
44             ];
45         auto &[ra, rb, rw] = edge[par[
46             now] ^ 1];
47         w -= mn[n];
48         rw += mn[n];
49         now = a;
50     }
51     return mn[n];
52 }
53 int max_flow_ans(){
54     int ans = 0;
55     while(true){
56         int t = bfs();
57         if(t == 0) break;
58         ans += t;
59     }
60     return ans;
61 }
62 };

```

4 Graph

4.1 bellman-ford

```

1 void bellman_ford(){
2     for(int i = 1; i <= n; i++){
3         bool update = false;
4         for(int j = 1; j <= n; j++){
5             for(auto k : g[j])

```

```

6         if(dis[k.F] > dis[j] + k.S)
7             update = true, dis[k.F]
8                 = dis[j] + k.S;
9         if(!update)break;
10        if(i == n)cout << "neg_cycle" << "\n";
11    }

```

4.2 dijk

```

1 priority_queue<pii, vector<pii>, greater<pii>
2 >> q;
3 q.push({0, 1});
4 dis[1] = 0;
5 while(q.size()){
6     auto [d, x] = q.top(); q.pop();
7     if(d > dis[x]) continue;
8     for(auto [to, w] : g[x]){
9         if(dis[to] > dis[x] + w){
10             dis[to] = dis[x] + w;
11             q.push({dis[to], to});
12         }
13     }

```

4.3 Euler-Path

```

1 void dfs(int x){
2     while(g[x].size()){
3         int nxt = g[x].back();
4         g[x].pop_back();
5         dfs(nxt);
6     }
7     ans.pb(x); // 倒序記錄答案
8 }

```

4.4 SCC

```

1 struct SCC{
2     vector<vector<int>> g, rg;
3     vector<bool> vis, used;
4     vector<int> component, st;
5     int component_cnt = 0, n;
6     void dfs1(int node){
7         vis[node] = true;
8         for(int i : g[node]) if(!vis[i])
9             dfs1(i);
10        st.pb(node);
11    }
12    void dfs2(int node){
13        used[node] = true;
14        for(int i : rg[node]) if(!used[i])
15            dfs2(i);
16        component[node] = component_cnt;
17    }

```

```

16 SCC(const vector<vector<int>> v){
17     n = v.size() - 1;
18     g = v;
19     rg.resize(n + 1);
20     vis.resize(n + 1, false);
21     used.resize(n + 1, false);
22     component.resize(n + 1, 0);
23     for(int i = 1; i <= n; i++) for(int
24         j : v[i]) rg[j].pb(i);
25 }
26 vector<int> find_component(){
27     for(int i = 1; i <= n; i++) if(!vis[
28         i]) dfs1(i);
29     reverse(st.begin(), st.end());
30     for(int i : st) if(!used[i]){
31         component_cnt++;
32         dfs2(i);
33     }
34 }

```

5 Linear Programming

5.1 simplex

```

1 /*target:
2 max \sum_{j=1}^n A_{0,j}*x_j
3 condition:
4 \sum_{j=1}^n A_{i,j}*x_j <= A_{i,0} /i=1~m
5 x_j >= 0 /j=1~n
6 VDB = vector<double>*/
7 template<class VDB>
8 VDB simplex(int m,int n,vector<VDB> a){
9     vector<int> left(m+1), up(n+1);
10    iota(left.begin(), left.end(), n);
11    iota(up.begin(), up.end(), 0);
12    auto pivot = [&](int x, int y){
13        swap(left[x], up[y]);
14        auto k = a[x][y]; a[x][y] = 1;
15        vector<int> pos;
16        for(int j = 0; j <= n; j++){
17            a[x][j] /= k;
18            if(a[x][j] != 0) pos.push_back(j);
19        }
20        for(int i = 0; i <= m; i++){
21            if(a[i][y]==0 || i == x) continue;
22            k = a[i][y], a[i][y] = 0;
23            for(int j : pos) a[i][j] -= k*a[x][j];
24        }
25    };
26    for(int x,y;;){
27        for(int i=x+1; i <= m; i++)
28            if(a[i][0]<a[x][0]) x = i;
29        if(a[x][0]>=0) break;
30        for(int j=y+1; j <= n; j++){
31            if(a[x][j]<a[x][y]) y = j;
32            if(a[x][y]==0) return VDB();//infeasible
33            pivot(x, y);
34        }
35        for(int x,y;;){

```

```

36         for(int j=y+1; j <= n; j++)
37             if(a[0][j] > a[0][y]) y = j;
38         if(a[0][y]<=0) break;
39         x = -1;
40         for(int i=1; i<=m; i++) if(a[i][y] > 0)
41             if(x == -1 || a[i][0]/a[i][y]
42                 < a[x][0]/a[x][y]) x = i;
43         if(x == -1) return VDB();//unbounded
44         pivot(x, y);
45     }
46     VDB ans(n + 1);
47     for(int i = 1; i <= m; i++)
48         if(left[i] <= n) ans[left[i]] = a[i][0];
49     ans[0] = -a[0][0];
50     return ans;
51 }

```

6 Number Theory

6.1 basic

```

1 template<typename T>
2 void gcd(const T &a,const T &b,T &d,T &x,T &
3     y){
4     if(!b) d=a,x=1,y=0;
5     else gcd(b,a%b,d,y,x), y-=x*(a/b);
6 }
7 long long int phi[N+1];
8 void phiTable(){
9     for(int i=1;i<=N;i++)phi[i]=i;
10    for(int i=1;i<=N;i++)for(x=i*2;x<=N;x+=i)
11        phi[x]-=phi[i];
12 }
13 void all_divdown(const LL &n){ // all n/x
14     for(LL a=1;a<=n;a=n/(n/(a+1))) {
15         // dosomething;
16     }
17 }
18 const int MAXPRIME = 1000000;
19 int iscom[MAXPRIME], prime[MAXPRIME],
20     primecnt;
21 int phi[MAXPRIME], mu[MAXPRIME];
22 void sieve(void){
23     memset(iscom,0,sizeof(iscom));
24     primecnt = 0;
25     phi[1] = mu[1] = 1;
26     for(int i=2;i<MAXPRIME;i++) {
27         if(!iscom[i]) {
28             prime[primecnt++] = i;
29             mu[i] = -1;
30             phi[i] = i-1;
31         }
32         for(int j=0;j<primecnt;j++) {
33             int k = i * prime[j];
34             if(k>MAXPRIME) break;
35             iscom[k] = prime[j];
36             if(i%prime[j]==0) {
37                 mu[k] = 0;
38                 phi[k] = phi[i] * prime[j];
39                 break;
40             } else {

```

```

38         mu[k] = -mu[i];
39         phi[k] = phi[i] * (prime[j]-1);
40     }
41 }
42 }
43 }
44 }
45 bool g_test(const LL &g, const LL &p, const
46     vector<LL> &v) {
47     for(int i=0;i<v.size();i++)
48         if(modexp(g,(p-1)/v[i],p)!=1)
49             return false;
50     return true;
51 }
52 LL primitive_root(const LL &p) {
53     if(p==2) return 1;
54     vector<LL> v;
55     Factor(p-1,v);
56     v.erase(unique(v.begin(), v.end()), v.end
57         ());
58     for(LL g=2;g<p;g++)
59         if(g_test(g,p,v))
60             return g;
61     puts("primitive_root NOT FOUND");
62     return -1;
63 }
64 int Legendre(const LL &a, const LL &p) {
65     return modexp(a%p,(p-1)/2,p); }
66 }
67 LL inv(const LL &a, const LL &n) {
68     LL d,x,y;
69     gcd(a,n,d,x,y);
70     return d==1 ? (x+n)%n : -1;
71 }
72 int inv[maxN];
73 LL invtable(int n,LL P){
74     inv[1]=1;
75     for(int i=2;i<n;i++)
76         inv[i]=(P-(P/i))*inv[P%i]%P;
77 }
78 LL log_mod(const LL &a, const LL &b, const
79     LL &p) {
80     // a ^ x = b ( mod p )
81     int m=sqrt(p+.5), e=1;
82     LL v=inv(modexp(a,m,p), p);
83     map<LL,int> x;
84     x[1]=0;
85     for(int i=1;i<m;i++) {
86         e = LLMul(e,a,p);
87         if(!x.count(e)) x[e] = i;
88     }
89     for(int i=0;i<m;i++) {
90         if(x.count(b)) return i*m + x[b];
91         b = LLMul(b,v,p);
92     }
93     return -1;
94 }
95 LL Tonelli_Shanks(const LL &n, const LL &p)
96 {
97     // x^2 = n ( mod p )
98     if(n==0) return 0;
99     if(Legendre(n,p)!=1) while(1) { puts("SQRT
100         ROOT does not exist"); }

```

```

98 int S = 0;
99 LL Q = p-1;
100 while( !(Q&1) ) { Q>>=1; ++S; }
101 if(S==1) return modexp(n%p,(p+1)/4,p);
102 LL z = 2;
103 for(; Legendre(z,p)!=-1; ++z)
104 LL c = modexp(z,Q,p);
105 LL R = modexp(n%p,(Q+1)/2,p), t = modexp(n
    %p,Q,p);
106 int M = S;
107 while(1) {
108     if(t==1) return R;
109     LL b = modexp(c,1L<<(M-i-1),p);
110     R = LLMul(R,b,p);
111     t = LLMul( LLMul(b,b,p), t, p);
112     c = LLMul(b,b,p);
113     M = i;
114 }
115 return -1;
116 }
117
118 template<typename T>
119 T Euler(T n){
120     T ans=n;
121     for(T i=2;i*i<=n;++i){
122         if(n%i==0){
123             ans=ans/i*(i-1);
124             while(n%i==0)n/=i;
125         }
126     }
127     if(n>1)ans=ans/n*(n-1);
128     return ans;
129 }
130
131 //Chinese_remainder_theorem
132 template<typename T>
133 T pow_mod(T n,T k,T m){
134     T ans=1;
135     for(n=(n>m?n%m:n);k;k>>=1){
136         if(k&1)ans=ans*n%m;
137         n=n*n%m;
138     }
139     return ans;
140 }
141
142 template<typename T>
143 T crt(vector<T> &m,vector<T> &a){
144     T M=1,tM,ans=0;
145     for(int i=0;i<(int)m.size();++i)M*=m[i];
146     for(int i=0;i<(int)a.size();++i){
147         tM=M/m[i];
148         ans=(ans+(a[i]*tM%M)*pow_mod(tM,Euler(m[i])-1,m[i])%M)%M;
149         /*如果m[i]是質數·Euler(m[i])-1=m[i]-2·
150            就不用算Euler了*/
151     }
152     return ans;
153 }
154
155 //java code
156 //求sqrt(N)的連分數
157 public static void Pell(int n){
158     BigInteger N,p1,p2,q1,q2,a0,a1,a2,g1,g2,h1

```

```

159     a0=a1=BigInteger.valueOf((int)Math.sqrt
        (1.0*n));
160     BigInteger ans=a0.multiply(a0);
161     if(ans.equals(BigInteger.valueOf(n)){
162         System.out.println("No solution!");
163         return ;
164     }
165     while(true){
166         g2=a1.multiply(h1).subtract(g1);
167         h2=N.subtract(g2.pow(2)).divide(h1);
168         a2=g2.add(a0).divide(h2);
169         p=a1.multiply(p2).add(p1);
170         q=a1.multiply(q2).add(q1);
171         if(p.pow(2).subtract(N.multiply(q.pow
            (2))).compareTo(BigInteger.ONE)==0)
            break;
172         g1=g2;h1=h2;a1=a2;
173         p1=p2;p2=p;
174         q1=q2;q2=q;
175     }
176     System.out.println(p+" "+q);
177 }

```

6.2 bit set

```

1 void sub_set(int S){
2     int sub=S;
3     do{
4         //對某集合的子集合的處理
5         sub=(sub-1)&S;
6     }while(sub!=S);
7 }
8
9 void k_sub_set(int k,int n){
10    int comb=(1<<k)-1,S=1<<n;
11    while(comb<S){
12        //對大小為k的子集合的處理
13        int x=comb&-comb,y=comb+x;
14        comb=((comb&~y)/x>>1)|y;
15    }

```

6.3 cantor expansion

```

1 int factorial[MAXN];
2 void init(){
3     factorial[0]=1;
4     for(int i=1;i<=MAXN;++i)factorial[i]=
        factorial[i-1]*i;
5 }
6 int encode(const vector<int> &s){
7     int n=s.size(),res=0;
8     for(int i=0;i<n;++i){
9         int t=0;
10        for(int j=i+1;j<n;++j)
11            if(s[j]<s[i])++t;
12        res+=t*factorial[n-i-1];
13    }
14    return res;
15 }

```

```

16 vector<int> decode(int a,int n){
17     vector<int> res;
18     vector<bool> vis(n,0);
19     for(int i=n-1;i>=0;--i){
20         int t=a/factorial[i],j;
21         for(j=0;j<n;++j)
22             if(!vis[j]){
23                 if(t==0)break;
24                 --t;
25             }
26         res.push_back(j);
27         vis[j]=1;
28         a%=factorial[i];
29     }
30     return res;
31 }

```

6.4 FFT

```

1 template<typename T,typename VT=vector<
    complex<T> >>
2 struct FFT{
3     const T pi;
4     FFT(const T pi=acos((T)-1)):pi(pi){}
5     unsigned bit_reverse(unsigned a,int len){
6         a=((a&0x55555555)<<1)|((a&0xAAAAAAAA)>>1);
7         a=((a&0x33333333)<<2)|((a&0xCCCCCCCC)>>2);
8         a=((a&0xF0F0F0F0)<<4)|((a&0xFF0F0F0F)>>4);
9         a=((a&0xFF0F0F0F)<<8)|((a&0xFFFF0F0F)>>8);
10        a=((a&0xFF0F0F0F)<<16)|((a&0xFFFF0F0F)>>16);
11        return a>>(32-len);
12    }
13    void fft(bool is_inv,VT &in,VT &out,int N)
14    {
15        int bitlen=__lg(N),num=is_inv?-1:1;
16        for(int i=0;i<N;++i)out[bit_reverse(i,
            bitlen)]=in[i];
17        for(int step=2;step<=N;step<=1){
18            const int mh=step>>1;
19            for(int i=0;i<mh;++i){
20                complex<T> wi=exp(complex<T>(0,i*num
                    *pi/mh));
21                for(int j=i;j<N;j+=step){
22                    int k=j+mh;
23                    complex<T> u=out[j],t=wi*out[k];
24                    out[j]=u+t;
25                    out[k]=u-t;
26                }
27            }
28        }
29        if(is_inv)for(int i=0;i<N;++i)out[i]/=N;
30    };

```

6.5 find real root

```

1 // an*x^n + ... + a1x + a0 = 0;
2 int sign(double x){
3     return x < -eps ? -1 : x > eps;

```

```

4 }
5
6 double get(const vector<double>&coef, double
    x){
7     double e = 1, s = 0;
8     for(auto i : coef) s += i*e, e *= x;
9     return s;
10 }
11
12 double find(const vector<double>&coef, int n
    , double lo, double hi){
13     double sign_lo, sign_hi;
14     if( !(sign_lo = sign(get(coef,lo))) )
        return lo;
15     if( !(sign_hi = sign(get(coef,hi))) )
        return hi;
16     if(sign_lo * sign_hi > 0) return INF;
17     for(int stp = 0; stp < 100 && hi - lo >
        eps; ++stp){
18         double m = (lo+hi)/2.0;
19         int sign_mid = sign(get(coef,m));
20         if(!sign_mid) return m;
21         if(sign_lo*sign_mid < 0) hi = m;
22         else lo = m;
23     }
24     return (lo+hi)/2.0;
25 }
26
27 vector<double> cal(vector<double>coef, int n
    ){
28     vector<double>res;
29     if(n == 1){
30         if(sign(coef[1])) res.pb(-coef[0]/coef
            [1]);
31         return res;
32     }
33     vector<double>dcoef(n);
34     for(int i = 0; i < n; ++i) dcoef[i] = coef
        [i+1]*(i+1);
35     vector<double>droot = cal(dcoef, n-1);
36     droot.insert(droot.begin(), -INF);
37     droot.pb(INF);
38     for(int i = 0; i+1 < droot.size(); ++i){
39         double tmp = find(coef, n, droot[i],
            droot[i+1]);
40         if(tmp < INF) res.pb(tmp);
41     }
42     return res;
43 }
44
45 int main () {
46     vector<double>ve;
47     vector<double>ans = cal(ve, n);
48     // 視情況把答案 +eps · 避免 -0
49 }

```

6.6 FWT

```

1 vector<int> F_OR_T(vector<int> f, bool
    inverse){
2     for(int i=0; (2<<i)<=f.size(); ++i)
3         for(int j=0; j<f.size(); j+=2<<i)
4             for(int k=0; k<(1<<i); ++k)

```

```

5         f[j+k+(1<<i)] += f[j+k]*(inverse
           ?-1:1);
6     return f;
7 }
8 vector<int> rev(vector<int> A) {
9     for(int i=0; i<A.size(); i+=2)
10         swap(A[i],A[i^(A.size()-1)]);
11     return A;
12 }
13 vector<int> F_AND_T(vector<int> f, bool
    inverse){
14     return rev(F_OR_T(rev(f), inverse));
15 }
16 vector<int> F_XOR_T(vector<int> f, bool
    inverse){
17     for(int i=0; (2<<i)<=f.size(); ++i)
18         for(int j=0; j<f.size(); j+=2<<i)
19             for(int k=0; k<(1<<i); ++k){
20                 int u=f[j+k], v=f[j+k+(1<<i)];
21                 f[j+k+(1<<i)] = u-v, f[j+k] = u+v;
22             }
23     if(inverse) for(auto &a:f) a/=f.size();
24     return f;
25 }

```

6.7 LinearCongruence

```

1 pair<LL,LL> LinearCongruence(LL a[],LL b[],
    LL m[],int n) {
2     // a[i]*x = b[i] ( mod m[i] )
3     for(int i=0;i<n;++i) {
4         LL x, y, d = extgcd(a[i],m[i],x,y);
5         if(b[i]%d!=0) return make_pair(-1LL,0LL)
6         ;
7         m[i] /= d;
8         b[i] = LLmul(b[i]/d,x,m[i]);
9     }
10    LL lastb = b[0], lastm = m[0];
11    for(int i=1;i<n;++i) {
12        LL x, y, d = extgcd(m[i],lastm,x,y);
13        if((lastb-b[i])%d!=0) return make_pair
14            (-1LL,0LL);
15        lastb = LLmul((lastb-b[i])/d,x,(lastm/d)
16            )*m[i];
17        lastm = (lastm/d)*m[i];
18        lastb = (lastb+b[i])%lastm;
19    }
20    return make_pair(lastb<0?lastb+lastm:lastb
21        ,lastm);
22 }

```

6.8 Lucas

```

1 ll C(ll n, ll m, ll p){// n!/m!/(n-m)!
2     if(n<m) return 0;
3     return f[n]*inv(f[m],p)%p*inv(f[n-m],p)%p;
4 }
5 ll L(ll n, ll m, ll p){
6     if(!m) return 1;
7     return C(n%p,m%p,p)*L(n/p,m/p,p)%p;

```

```

8 }
9 ll Wilson(ll n, ll p){ // n!%p
10    if(!n)return 1;
11    ll res=Wilson(n/p, p);
12    if((n/p)%2) return res*(p-f[n%p])%p;
13    return res*f[n%p]%p; //(p-1)!%p=-1
14 }

```

6.9 Matrix

```

1 template<typename T>
2 struct Matrix{
3     using rt = std::vector<T>;
4     using mt = std::vector<rt>;
5     using matrix = Matrix<T>;
6     int r,c;
7     mt m;
8     Matrix(int r,int c):r(r),c(c),m(r,rt(c)){}
9     rt& operator[](int i){return m[i];}
10    matrix operator+(const matrix &a){
11        matrix rev(r,c);
12        for(int i=0;i<r;++i)
13            for(int j=0;j<c;++j)
14                rev[i][j]=m[i][j]+a.m[i][j];
15        return rev;
16    }
17    matrix operator-(const matrix &a){
18        matrix rev(r,c);
19        for(int i=0;i<r;++i)
20            for(int j=0;j<c;++j)
21                rev[i][j]=m[i][j]-a.m[i][j];
22        return rev;
23    }
24    matrix operator*(const matrix &a){
25        matrix rev(r,a.c);
26        matrix tmp(a.c,a.r);
27        for(int i=0;i<a.r;++i)
28            for(int j=0;j<a.c;++j)
29                tmp[j][i]=a.m[i][j];
30        for(int i=0;i<r;++i)
31            for(int j=0;j<a.c;++j)
32                for(int k=0;k<c;++k)
33                    rev.m[i][j]+=m[i][k]*tmp[j][k];
34        return rev;
35    }
36    bool inverse(){
37        Matrix t(r,r+c);
38        for(int y=0;y<r;y++){
39            t.m[y][c+y] = 1;
40            for(int x=0;x<c;++x)
41                t.m[y][x]=m[y][x];
42        }
43        if( !t.gas() )
44            return false;
45        for(int y=0;y<r;y++){
46            for(int x=0;x<c;++x)
47                m[y][x]=t.m[y][c+x]/t.m[y][y];
48            return true;
49        }
50    T gas(){
51        vector<T> lazy(r,1);
52        bool sign=false;
53        for(int i=0;i<r;++i){

```

```

54        if( m[i][i]==0 ){
55            int j=i+1;
56            while(j<r&&!m[j][i])j++;
57            if(j==r)continue;
58            m[i].swap(m[j]);
59            sign=!sign;
60        }
61        for(int j=0;j<r;++j){
62            if(i==j)continue;
63            lazy[j]=lazy[j]*m[i][i];
64            T mx=m[j][i];
65            for(int k=0;k<c;++k)
66                m[j][k]=m[j][k]*m[i][i]-m[i][k]*mx
67                ;
68        }
69        T det=sign?-1:1;
70        for(int i=0;i<r;++i){
71            det = det*m[i][i];
72            det = det/lazy[i];
73            for(auto &j:m[i])j/=lazy[i];
74        }
75        return det;
76    }
77 };

```

6.10 MillerRobin

```

1 ULL LLmul(ULL a, ULL b, const ULL &mod) {
2     LL ans=0;
3     while(b) {
4         if(b&1) {
5             ans+=a;
6             if(ans>=mod) ans-=mod;
7         }
8         a<<=1, b>>=1;
9         if(a>=mod) a-=mod;
10    }
11    return ans;
12 }
13 ULL mod_mul(ULL a,ULL b,ULL m){
14     a%=m,b%=m; /* fast for m < 2^58 */
15     ULL y=(ULL)((double)a*b/m+0.5);
16     ULL r=(a*b-y*m)%m;
17     return r<0?r+m:r;
18 }
19 template<typename T>
20 T pow(T a,T b,T mod){//a^b%mod
21     T ans=1;
22     for(;b;a=mod_mul(a,a,mod),b>>=1)
23         if(b&1)ans=mod_mul(ans,a,mod);
24     return ans;
25 }
26 int sprp[3]={2,7,61}; //int範圍可解
27 int llsprp
    [7]={2,325,9375,28178,450775,9780504,
28     1795265022}; //至少unsigned long long範圍
29 template<typename T>
30 bool isprime(T n,int *sprp,int num){
31     if(n==2)return 1;
32     if(n<2||n%2==0)return 0;
33     int t=0;

```

```

34     T u=n-1;
35     for(;u%2==0;++t)u>>=1;
36     for(int i=0;i<num;++i){
37         T a=sprp[i]%n;
38         if(a==0||a==1||a==n-1)continue;
39         T x=pow(a,u,n);
40         if(x==1||x==n-1)continue;
41         for(int j=0;j<t;++j){
42             x=mod_mul(x,x,n);
43             if(x==1)return 0;
44             if(x==n-1)break;
45         }
46         if(x==n-1)continue;
47         return 0;
48     }
49     return 1;
50 }

```

6.11 NTT

```

1 2615053605667*(2^18)+1,3
2 15*(2^27)+1,31
3 479*(2^21)+1,3
4 7*17*(2^23)+1,3
5 3*3*211*(2^19)+1,5
6 25*(2^22)+1,3
7 template<typename T,typename VT=vector<T> >
8 struct NTT{
9     const T P,G;
10    NTT(T p=(1<<23)*7*17+1,T g=3):P(p),G(g){}
11    unsigned bit_reverse(unsigned a,int len){
12        //Look FFT.cpp
13    }
14    T pow_mod(T n,T k,T m){
15        T ans=1;
16        for(n=(n>=m?n%m:n);k;k>>=1){
17            if(k&1)ans=ans*n%m;
18            n=n*n%m;
19        }
20        return ans;
21    }
22    void ntt(bool is_inv,VT &in,VT &out,int N)
23    {
24        int bitlen=__lg(N);
25        for(int i=0;i<N;++i)out[bit_reverse(i,
26            bitlen)]=in[i];
27        for(int step=2,id=1;step<=N;step<<=1,++
28            id){
29            T wn=pow_mod(G,(P-1)>>id,P),wi=1,u,t;
30            const int mh=step>>1;
31            for(int i=0;i<mh;++i){
32                for(int j=i;j<N;j+=step){
33                    u=out[j],t=wi*out[j+mh]%P;
34                    out[j]=u+t;
35                    out[j+mh]=u-t;
36                    if(out[j]>=P)out[j]-=P;
37                    if(out[j+mh]<0)out[j+mh]+=P;
38                }
39                wi=wi*wn%P;
40            }
41        }
42        if(is_inv){

```



```

40     for(int i=1;i<N/2;++i)swap(out[i],out[
        N-i]);
41     T invn=pow_mod(N,P-2,P);
42     for(int i=0;i<N;++i)out[i]=out[i]*invn
        %P;
43 }
44 }
45 };

```

6.12 Simpson

```

1 double simpson(double a,double b){
2     double c=a+(b-a)/2;
3     return (F(a)+4*F(c)+F(b))*(b-a)/6;
4 }
5 double asr(double a,double b,double eps,
6     double A){
7     double c=a+(b-a)/2;
8     double L=simpson(a,c),R=simpson(c,b);
9     if( abs(L+R-A)<15*eps )
10        return L+R+(L+R-A)/15.0;
11    return asr(a,c,eps/2,L)+asr(c,b,eps/2,R);
12 }
13 double asr(double a,double b,double eps){
14    return asr(a,b,eps,simpson(a,b));
15 }

```

6.13 外星模運算

```

1 //a[0]^a[1]^a[2]^...
2 #define maxn 1000000
3 int euler[maxn+5];
4 bool is_prime[maxn+5];
5 void init_euler(){
6     is_prime[1]=1;//一不是質數
7     for(int i=1;i<=maxn;i++)euler[i]=i;
8     for(int i=2;i<=maxn;i++){
9         if(!is_prime[i]){//是質數
10            euler[i]--;
11            for(int j=i<1;j<=maxn;j+=i){
12                is_prime[j]=1;
13                euler[j]=euler[j]/i*(i-1);
14            }
15        }
16    }
17 }
18 LL pow(LL a,LL b,LL mod){//a^b%mod
19     LL ans=1;
20     for(;b;a=a%mod,b>=1)
21         if(b&1)ans=ans*a%mod;
22     return ans;
23 }
24 bool isless(LL *a,int n,int k){
25     if(*a==1)return k>1;
26     if(--n==0)return *a<k;
27     int next=0;
28     for(LL b=1;b<k;++next)
29         b*=*a;
30     return isless(a+1,n,next);

```

```

31 }
32 LL high_pow(LL *a,int n,LL mod){
33     if(*a==1||--n==0)return *a%mod;
34     int k=0,r=euler[mod];
35     for(LL tma=1;tma!=pow(*a,k+r,mod);++k)
36         tma=tma*(a%mod);
37     if(isless(a+1,n,k))return pow(*a,high_pow(
        a+1,n,k),mod);
38     int tmd=high_pow(a+1,n,r), t=(tmd-k+r)%r;
39     return pow(*a,k+t,mod);
40 }
41 LL a[1000005];
42 int t,mod;
43 int main(){
44     init_euler();
45     scanf("%d",&t);
46     #define n 4
47     while(t--){
48         for(int i=0;i<n;++i)scanf("%Lld",&a[i]);
49         scanf("%d",&mod);
50         printf("%Lld\n",high_pow(a,n,mod));
51     }
52     return 0;
53 }

```

6.14 數位統計

```

1 ll d[65], dp[65][2];//up區間是不是完整
2 ll dfs(int p,bool is8,bool up){
3     if(!p)return 1; // 回傳0是不是答案
4     if(!up&&~dp[p][is8])return dp[p][is8];
5     int mx = up?d[p]:9;//可以用的有那些
6     ll ans=0;
7     for(int i=0;i<=mx;++i){
8         if( is8&&i==7 )continue;
9         ans += dfs(p-1,i==8,up&&i==mx);
10    }
11    if(!up)dp[p][is8]=ans;
12    return ans;
13 }
14 ll f(ll N){
15     int k=0;
16     while(N){ // 把數字先分解到陣列
17         d[++k] = N%10;
18         N/=10;
19     }
20     return dfs(k,false,true);
21 }

```

6.15 質因數分解

```

1 LL func(const LL n,const LL mod,const int c)
2 {
3     return (LLmul(n,n,mod)+c+mod)%mod;
4 }
5 LL pollorroho(const LL n, const int c) { //循
    環節長度
6     LL a=1, b=1;

```

```

7     a=func(a,n,c)%n;
8     b=func(b,n,c)%n; b=func(b,n,c)%n;
9     while(gcd(abs(a-b),n)==1) {
10        a=func(a,n,c)%n;
11        b=func(b,n,c)%n; b=func(b,n,c)%n;
12    }
13    return gcd(abs(a-b),n);
14 }
15 void prefactor(LL &n, vector<LL> &v) {
16     for(int i=0;i<12;++i) {
17         while(n%prime[i]==0) {
18             v.push_back(prime[i]);
19             n/=prime[i];
20         }
21     }
22 }
23 void smallfactor(LL n, vector<LL> &v) {
24     if(n<MAXPRIME) {
25         while(isp[(int)n]) {
26             v.push_back(isp[(int)n]);
27             n/=isp[(int)n];
28         }
29         v.push_back(n);
30     } else {
31         for(int i=0;i<primecnt&&prime[i]*prime[i]
32             ]<=n;++i) {
33             while(n%prime[i]==0) {
34                 v.push_back(prime[i]);
35                 n/=prime[i];
36             }
37         }
38         if(n!=1) v.push_back(n);
39     }
40 }
41 void comfactor(const LL &n, vector<LL> &v) {
42     if(n<1e9) {
43         smallfactor(n,v);
44         return;
45     }
46     if(Isprime(n)) {
47         v.push_back(n);
48         return;
49     }
50     LL d;
51     for(int c=3;c<=n; c+=2) {
52         d = pollorroho(n,c);
53         if(d!=n) break;
54     }
55     comfactor(d,v);
56     comfactor(n/d,v);
57 }
58 void Factor(const LL &x, vector<LL> &v) {
59     LL n = x;
60     if(n==1) { puts("Factor 1"); return; }
61     prefactor(n,v);
62     if(n==1) return;
63     comfactor(n,v);
64     sort(v.begin(),v.end());
65 }
66 void AllFactor(const LL &n,vector<LL> &v) {
67     vector<LL> tmp;

```

```

72     Factor(n,tmp);
73     v.clear();
74     v.push_back(1);
75     int len;
76     LL now=1;
77     for(int i=0;i<tmp.size();++i) {
78         if(i==0 || tmp[i]!=tmp[i-1]) {
79             len = v.size();
80             now = 1;
81         }
82         now*=tmp[i];
83         for(int j=0;j<len;++j)
84             v.push_back(v[j]*now);
85     }
86 }

```

7 String

7.1 ac 自動機

```

1 int trie[(int)5e5 + 5][26], node_cnt = 0,
2     fail[(int)5e5 + 5]; // ac_automachine
3 int pattern_end[(int)5e5 + 5];
4 void insert(string s, int id)
5 {
6     int u = 0;
7     for(char c : s){
8         int v = c - 'a';
9         if(!trie[u][v]) trie[u][v] = ++
10            node_cnt;
11        u = trie[u][v];
12    }
13    pattern_end[id] = u;
14 }
15 void build_ac_automachine()
16 {
17     //bfs
18     queue<int> q;
19     for(int i = 0; i < 26; i++) if(trie[0][i]
20        ] > 0) q.push(trie[0][i]);
21     while(q.size()){
22         int x = q.front(); q.pop();
23         for(int i = 0; i < 26; i++){
24             if(trie[x][i] > 0){
25                 fail[trie[x][i]] = trie[fail
26                    [x]][i];
27                 q.push(trie[x][i]);
28             }
29         }
30     }
31 }

```

7.2 KMP

```

1 vector<int> pi(s.size(), 0);
2 for(int i = 1, j = 0; i < s.size(); i++){

```

```

3 while(j > 0 and s[i] != s[j]) j = pi[j - 1];
4 if(s[i] == s[j]) j++;
5 pi[i] = j;
6 }

```

7.3 manacher

```

1 string s, t = "^";
2 cin >> s;
3 for(char c : s){
4     t += "#";
5     t += c;
6 }
7 t += "##";
8 vector<int> p(t.size(), 0);
9 int c = 0, r = 0, len = 0, center = 0;
10 for(int i = 1; i < t.size() - 1; i++){
11     if(i < r) p[i] = min(r - i, p[2 * c - i]);
12     while(t[i + p[i] + 1] == t[i - p[i] - 1]) p[i]++;
13     if(i + p[i] > r){
14         r = i + p[i];
15         c = i;
16     }
17     if(p[i] > len){
18         len = p[i];
19         center = i;
20     }
21 }
22 cout << s.substr((center - len) / 2, len) <<
    "\n"; //Longest Palindrome

```

7.4 suffix-array

```

1 vector<int> buildSuffixArray(string s, int n)
2 {
3     vector<int> sa(n), tmp(n), rank(n);
4     /*
5     sa[i]: 從sa[i]開始的後綴字串是第i小
6     rank[i]: 從i開始的後綴字串是第幾小
7     tmp: rank的暫存
8     */
9     for(int i = 0; i < n; i++){ // 初始化長度為1的後綴字串
10         sa[i] = i;
11         rank[i] = s[i] - 'a';
12     }
13     for(int k = 1; k < n; k <= 1){
14         auto cmp = [&](int i, int j){ //比較長度為2*k的字串
15             if(rank[i] != rank[j]) return rank[i] < rank[j];
16             int ri = i + k < n ? rank[i + k] : -1;
17             int rj = j + k < n ? rank[j + k] : -1;

```

```

18         return ri < rj;
19     };
20     sort(sa.begin(), sa.end(), cmp); // sa 按照字典序(rank)排列
21     /*
22     更新rank
23     要先用tmp存，因為cmp會用到rank的舊資料
24     */
25     tmp[sa[0]] = 0;
26     for(int i = 1; i < n; i++) tmp[sa[i]] = tmp[sa[i - 1]] + (cmp(sa[i - 1], sa[i]) ? 1 : 0);
27     rank = tmp;
28 }
29 return sa;
30 }
31 vector<int> buildLCP(string s, vector<int> sa, int n)
32 {
33     vector<int> rank(n, 0); // rank[i]: 從sa[i]開始的字尾字串按照字典序排序後的位置
34     for(int i = 0; i < n; i++) rank[sa[i]] = i;
35     int h = 0; // 因為是按字串s原始位置順序計算，用前綴相似的特性，h每次最多只會減少1
36     vector<int> lcp(n, 0); // 與字典序前一個的字尾字串的最長共同前綴長度
37     for(int i = 0; i < n; i++){
38         if(rank[i] > 0){
39             int j = sa[rank[i] - 1]; // 字典序前一個的字尾字串
40             while(i + h < n and j + h < n and s[i + h] == s[j + h]) h++;
41             lcp[rank[i]] = h;
42             if(h > 0) h--;
43         }
44     }
45     return lcp;
46 }
47 int ans = 0, n = s.size();
48 // ans: the number of distinct substrings that appear in a string
49 for(int i = 0; i < n; i++) ans += n - sa[i] - lcp[i];
50 /*
51 (n - sa[i]): 這個字尾本來可以貢獻這麼多個子字串。
52 lcp[i]: 扣掉跟字典序前一個字尾長得一模一樣的前綴，因為那些已經算過了。
53 */

```

8 Tarjan

8.1 dominator tree

```

1 struct dominator_tree{
2     static const int MAXN=5005;
3     int n; // 1-base
4     vector<int> G[MAXN], rG[MAXN];
5     int pa[MAXN], dfn[MAXN], id[MAXN], dfnCt;
6     int semi[MAXN], idom[MAXN], best[MAXN];
7     vector<int> tree[MAXN]; // tree here
8     void init(int _n){
9         n = _n;
10        for(int i=1; i<=n; ++i)
11            G[i].clear(), rG[i].clear();
12    }
13    void add_edge(int u, int v){
14        G[u].push_back(v);
15        rG[v].push_back(u);
16    }
17    void dfs(int u){
18        id[dfnCt]=++dfnCt=u;
19        for(auto v:G[u]) if(!dfn[v])
20            dfs(v),pa[dfn[v]]=dfn[u];
21    }
22    int find(int y,int x){
23        if(y <= x) return y;
24        int tmp = find(pa[y],x);
25        if(semi[best[y]] > semi[best[pa[y]]])
26            best[y] = best[pa[y]];
27        return pa[y] = tmp;
28    }
29    void tarjan(int root){
30        dfnCt = 0;
31        for(int i=1; i<=n; ++i){
32            dfn[i] = idom[i] = 0;
33            tree[i].clear();
34            best[i] = semi[i] = i;
35        }
36        dfs(root);
37        for(int i=dfnCt; i>1; --i){
38            int u = id[i];
39            for(auto v:rG[u]) if(v=dfn[v]){
40                find(v,i);
41                semi[i]=min(semi[i],semi[best[v]]);
42            }
43            tree[semi[i]].push_back(i);
44            for(auto v:tree[pa[i]]){
45                find(v, pa[i]);
46                idom[v] = semi[best[v]]==pa[i] ? pa[i] : best[v];
47            }
48            tree[pa[i]].clear();
49        }
50        for(int i=2; i<=dfnCt; ++i){
51            if(idom[i] != semi[i])
52                idom[i] = idom[idom[i]];
53            tree[id[idom[i]]].push_back(id[i]);
54        }
55    }
56 }
57 }dom;

```

8.2 tnfshb017 2 sat

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define MAXN 8001

```

```

4 #define MAXN2 MAXN*4
5 #define n(X) ((X)+2*N)
6 vector<int> v[MAXN2], rv[MAXN2], vis_t;
7 int N,M;
8 void addedge(int s,int e){
9     v[s].push_back(e);
10    rv[e].push_back(s);
11 }
12 int scc[MAXN2];
13 bool vis[MAXN2]={false};
14 void dfs(vector<int> *uv,int n,int k=-1){
15     vis[n]=true;
16     for(int i=0;i<uv[n].size();++i)
17         if(!vis[uv[n][i]])
18             dfs(uv,uv[n][i],k);
19     if(uv==v)vis_t.push_back(n);
20     scc[n]=k;
21 }
22 void solve(){
23     for(int i=1;i<=N;++i){
24         if(!vis[i])dfs(v,i);
25         if(!vis[n(i)])dfs(v,n(i));
26     }
27     memset(vis,0,sizeof(vis));
28     int c=0;
29     for(int i=vis_t.size()-1;i>=0;--i)
30         if(!vis[vis_t[i]])
31             dfs(rv,vis_t[i],c++);
32 }
33 int main(){
34     int a,b;
35     scanf("%d%d",&N,&M);
36     for(int i=1;i<=N;++i){
37         // (A or B)&(!A & !B) A^B
38         a=i*2-1;
39         b=i*2;
40         addedge(n(a),b);
41         addedge(n(b),a);
42         addedge(a,n(b));
43         addedge(b,n(a));
44     }
45     while(M--){
46         scanf("%d%d",&a,&b);
47         a = a>0?a*2-1:-a*2;
48         b = b>0?b*2-1:-b*2;
49         // A or B
50         addedge(n(a),b);
51         addedge(n(b),a);
52     }
53     solve();
54     bool check=true;
55     for(int i=1;i<=2*N;++i)
56         if(scc[i]==scc[n(i)])
57             check=false;
58     if(check){
59         printf("%d\n",N);
60         for(int i=1;i<=2*N;i+=2){
61             if(scc[i]>scc[i+2*N]) putchar('+');
62             else putchar('-');
63         }
64         puts("");
65     }else puts("0");
66     return 0;
67 }

```

8.3 橋連通分量

```

1 #define N 1005
2 struct edge{
3     int u,v;
4     bool is_bridge;
5     edge(int u=0,int v=0):u(u),v(v),is_bridge
6         (0){}
7 };
8 vector<edge> E;
9 vector<int> G[N]; // 1-base
10 int low[N],vis[N],Time;
11 int bcc_id[N],bridge_cnt,bcc_cnt; // 1-base
12 int st[N],top; // BCC用
13 void add_edge(int u,int v){
14     G[u].push_back(E.size());
15     E.emplace_back(u,v);
16     G[v].push_back(E.size());
17     E.emplace_back(v,u);
18 }
19 void dfs(int u,int re=-1){ // u當前點, re為u連
20     接前一個點的邊
21     int v;
22     low[u]=vis[u]++;Time;
23     st[top++]=u;
24     for(int e:G[u]){
25         v=E[e].v;
26         if(!vis[v]){
27             dfs(v,e^1); // e^1 反向邊
28             low[u]=min(low[u],low[v]);
29             if(vis[u]<low[v]){
30                 E[e].is_bridge=E[e^1].is_bridge=1;
31                 ++bridge_cnt;
32             }
33             }else if(vis[v]<vis[u]&&e!=re){
34                 low[u]=min(low[u],vis[v]);
35             }
36         }
37     }
38 }
39 void bcc_init(int n){
40     Time=bcc_cnt=bridge_cnt=top=0;
41     E.clear();
42     for(int i=1;i<=n;++i){
43         G[i].clear();
44         vis[i]=bcc_id[i]=0;
45     }
46 }
47 }

```

8.4 雙連通分量 & 割點

```

1 #define N 1005
2 vector<int> G[N]; // 1-base
3 vector<int> bcc[N]; // 存每塊雙連通分量的點
4 int low[N],vis[N],Time;
5 int bcc_id[N],bcc_cnt; // 1-base
6 bool is_cut[N]; // 是否為割點

```

```

7 int st[N],top;
8 void dfs(int u,int pa=-1){ // u當前點, pa父親
9     int t, child=0;
10    low[u]=vis[u]++;Time;
11    st[top++]=u;
12    for(int v:G[u]){
13        if(!vis[v]){
14            dfs(v,u),++child;
15            low[u]=min(low[u],low[v]);
16            if(vis[u]<=low[v]){
17                is_cut[u]=1;
18                bcc[++bcc_cnt].clear();
19                do{
20                    bcc_id[t=st[--top]]=bcc_cnt;
21                    bcc[bcc_cnt].push_back(t);
22                }while(t!=v);
23                bcc_id[u]=bcc_cnt;
24                bcc[bcc_cnt].push_back(u);
25            }
26            }else if(vis[v]<vis[u]&&v!=pa){ // 反向邊
27                low[u] = min(low[u],vis[v]);
28            } // u是dfs樹的根要特判
29            if(pa==-1&&child<2)is_cut[u]=0;
30        }
31    }
32    void bcc_init(int n){
33        Time=bcc_cnt=top=0;
34        for(int i=1;i<=n;++i){
35            G[i].clear();
36            is_cut[i]=vis[i]=bcc_id[i]=0;
37        }
38    }
39 }

```

9 Tree Problem

9.1 find-centroid

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 #define int long long
4 #define KCC ios::sync_with_stdio(0);cin.tie
5     (0);
6 #define pb push_back
7 int subtree[(int)2e5 + 5], n;
8 vector<int> g[(int)2e5 + 5];
9 void dfs(int now, int par)
10 {
11     subtree[now] = 1;
12     for(auto w : g[now]){
13         if(w != par){
14             dfs(w, now);
15             subtree[now] += subtree[w];
16         }
17     }
18     return ;
19 }
20 int get(int now, int par)
21 {
22     for(auto w : g[now]){
23         if(w != par and subtree[w] * 2 > n)
24             return get(w, now);
25     }
26 }

```

```

24 }
25 return now;
26 }
27 signed main()
28 {
29     KCC
30     cin >> n;
31     for(int i = 0; i < n - 1; i++){
32         int a, b; cin >> a >> b;
33         g[a].pb(b);
34         g[b].pb(a);
35     }
36     dfs(1, 0);
37     cout << get(1, 0) << "\n";
38     return 0;
39 }

```

9.2 hld

```

1 vector<int> edge[MAXN];
2 int ar[MAXN], sz[MAXN], par[MAXN], son[MAXN]
3     , dep[MAXN], dfn[MAXN], top[MAXN], t =
4     0;
5 void dfs_son(int x, int p, int d)
6 {
7     sz[x] = 1;
8     dep[x] = d;
9     son[x] = 0;
10    for(int i : edge[x]){
11        if(i != p){
12            dfs_son(i, x, d + 1);
13            sz[x] += sz[i];
14            par[i] = x;
15            if(sz[i] > sz[son[x]]) son[x] =
16                i;
17        }
18    }
19 }
20 void dfs_hld(int x, int p)
21 {
22     dfn[x] = ++t;
23     if(son[x] != 0){
24         top[son[x]] = top[x];
25         dfs_hld(son[x], x);
26     }
27     for(int i : edge[x]){
28         if(i != p and i != son[x]){
29             top[i] = i;
30             dfs_hld(i, x);
31         }
32     }
33 }
34 int main()
35 {
36     int n, q;
37     cin >> n >> q;
38     for(int i = 1; i <= n; i++) cin >> ar[i]
39         ;
40     for(int i = 0; i < n - 1; i++){
41         int u, v;
42         cin >> u >> v;
43         edge[u].pb(v);
44         edge[v].pb(u);
45     }
46 }

```

```

41 }
42 dfs_son(1, 0, 1);
43 top[1] = 1;
44 dfs_hld(1, 0);
45
46 vector<int> v(n + 5);
47 for(int i = 1; i <= n; i++) v[dfn[i]] =
48     ar[i];
49 seg_tree seg(n);
50 seg.build(v, 1, n, 1);
51 while(q--){
52     int op; cin >> op;
53     if(op == 1){ // change the value of
54         node s to x
55         int s, x;
56         cin >> s >> x;
57         seg.upd(dfn[s], x, 1, n, 1);
58     }
59     else{ // find the maximum value on
60         the path between nodes a and b.
61         int a, b, ans = 0;
62         cin >> a >> b;
63         while(top[a] != top[b]){
64             if(dfn[top[a]] < dfn[top[b]]
65                 swap(a, b);
66             ans = max(ans, seg.query(dfn
67                 [top[a]], dfn[a], 1, n,
68                 1));
69             a = par[top[a]];
70         }
71         if(dfn[a] > dfn[b]) swap(a, b);
72         ans = max(ans, seg.query(dfn[a],
73             dfn[b], 1, n, 1));
74         cout << ans << " ";
75     }
76 }
77 }

```

9.3 lca

```

1 void dfs(int now, int par)
2 {
3     deep[now] = deep[par] + 1;
4     p[now][0] = par;
5     for(int i : v[now]) if(i != par) dfs(i,
6         now);
7     return;
8 }
9 int lca(int u, int v)
10 {
11     int x = deep[u], y = deep[v];
12     if(x > y){
13         swap(u, v);
14         swap(x, y);
15     }
16     y -= x; // x < y
17     for(int i = 25; i >= 0; i--) if(y & (1 <<
18         i)) v = p[v][i];
19     if(u == v) return u;
20     for(int i = 25; i >= 0; i--){
21         if(p[u][i] != p[v][i]){
22             u = p[u][i];
23             v = p[v][i];
24         }
25     }
26     return p[u][0];
27 }

```



```

22     }
23 }
24 return p[u][0];
25 }
26 for(int j = 1; j < 30; j++){
27     for(int i = 2; i <= n; i++){
28         p[i][j] = p[p[i][j - 1]][j - 1];
29     }
30 }

```

10 compare

10.1 check

```

1 #include<bits/stdc++.h>
2 using namespace std;
3 int main() {
4     // 1. 在迴圈外編譯，並檢查是否成功
5     cout << "Compiling files..." << endl;
6
7     // system() 回傳 0 代表成功執行
8     int c1 = system("g++ -Wall -std=c++20
9         sol.cpp -o sol.exe");
10    int c2 = system("g++ -Wall -std=c++20 bf
11        .cpp -o bf.exe");
12    int c3 = system("g++ -Wall -std=c++20
13        gen.cpp -o gen.exe");
14
15    if (c1 != 0 || c2 != 0 || c3 != 0) {
16        cout << "Compilation Failed! Check
17            if sol.cpp, bf.cpp, and gen.cpp
18            exist." << endl;
19        return 1;
20    }
21
22    cout << "Compilation successful.
23        Starting stress test..." << endl;
24
25    for (int T = 1; ; T++) {
26        // 2. 執行程序
27        system("gen.exe > input.txt");
28        system("sol.exe < input.txt > sol.
29            out");
30        system("bf.exe < input.txt > bf.out"
31            );
32
33        // 3. 使用 fc /W 比對
34        if (system("fc sol.out bf.out > nul"
35            )) {
36            system("fc sol.out bf.out");
37            cout << "WA on Test #" << T << "
38                !" << endl;
39
40            // 發生錯誤時停止，此時 input.
41            txt 就是出錯的測資
42            break;
43        } else {
44            if (T % 10 == 0) cout << "Passed
45                " << T << " tests..." <<
46                endl;
47        }
48    }
49 }

```

```

34     }
35     return 0;
36 }

```

10.2 gen

```

1 #include<bits/stdc++.h>
2 using namespace std;
3
4 // 使用隨機種子，確保每次生成的數據都不同
5 mt19937_64 rng(chrono::steady_clock::now().
6     time_since_epoch().count());
7
8 // 生成 [L, r] 範圍內的隨機整數
9 long long rnd(long long l, long long r) {
10     return uniform_int_distribution<long
11         long>(l, r)(rng);
12 }
13
14 // 生成一棵隨機樹 (N 個點，N-1 條邊)
15 void gen_tree(int n) {
16     for (int i = 2; i <= n; i++) {
17         // 讓每個點 i 連向 [1, i-1] 中的隨機
18         // 點
19         int u = i;
20         int v = rnd(1, i - 1);
21         cout << u << " " << v << " \n";
22     }
23 }

```

11 default

11.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4         __PRETTY_FUNCTION__, __LINE__, #
5         __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8
9 template<typename I> void _DO(I&&x){cerr<<x
10     <<endl;}
11
12 template<typename I,typename...T> void _DO(I
13     &&x,T&&...tail){cerr<<x<<" ";_DO(tail
14     ...);}
15
16 #else
17 #define dbg(...)
18 #endif

```

11.2 ext

```

1 #include<bits/extc++.h>
2 #include<ext/pd_ds/assoc_container.hpp>
3 #include<ext/pd_ds/tree_policy.hpp>
4 using namespace __gnu_cxx;
5 using namespace __gnu_pbds;
6 template<typename T>
7 using pbds_set = tree<T,null_type,less<T>,
8     rb_tree_tag,
9     tree_order_statistics_node_update>;
10 template<typename T,typename U>
11 using pbds_map = tree<T,U,less<T>,
12     rb_tree_tag,
13     tree_order_statistics_node_update>;
14 using heap=__gnu_pbds::priority_queue<int>;
15 //s.find_by_order(1);//0 base
16 //s.order_of_key(1);

```

11.3 IncStack

```

1 //Magic
2 #pragma GCC optimize "Ofast"
3 //stack resize,change esp to rsp if 64-bit
4 system
5 asm("mov %0,%esp\n" ::"g"(mem+1000000));
6 -Wl,--stack,214748364 -trigraphs
7 #pragma comment(linker, "/STACK
8     :1024000000,1024000000")
9 //linux stack resize
10 #include<sys/resource.h>
11 void increase_stack(){
12     const rlim_t ks=64*1024*1024;
13     struct rlimit rl;
14     int res=getrlimit(RLIMIT_STACK,&rl);
15     if(!res&&rl.rlim_cur<ks){
16         rl.rlim_cur=ks;
17         res=setrlimit(RLIMIT_STACK,&rl);
18     }
19 }

```

11.4 input

```

1 inline int read(){
2     int x=0; bool f=0; char c=getchar();
3     while(ch<'0' || '9'<ch)f|=ch=='-',ch=getchar
4         ();
5     while('0'<=ch&&ch<='9')x=x*10-'0'+ch,ch=
6         getchar();
7     return f?-x:x;
8 }
9 // #!/bin/bash
10 // g++ -std=c++11 -O2 -Wall -Wextra -Wno-
11     unused-result -DDEBUG $1 && ./a.out
12 // -fsanitize=address -fsanitize=undefined
13     -fsanitize=return

```

12 other

12.1 WhatDay

```

1 int whatday(int y,int m,int d){
2     if(m<=2)m+=12,--y;
3     if(y<1752||y==1752&&m<9||y==1752&&m==9&&d
4         <3)
5         return (d+2*m+3*(m+1)/5+y+y/4+5)%7;
6     return (d+2*m+3*(m+1)/5+y+y/4-y/100+y/400)
7         %7;
8 }

```

12.2 上下最大正方形

```

1 void solve(int n,int a[],int b[]){// 1-base
2     int ans=0;
3     deque<int>da,db;
4     for(int l=1,r=1;r<=n;++r){
5         while(da.size()&&a[da.back()]>=a[r]){
6             da.pop_back();
7         }
8         da.push_back(r);
9         while(db.size()&&b[db.back()]>=b[r]){
10             db.pop_back();
11         }
12         db.push_back(r);
13         for(int d=a[da.front()]+b[db.front()];r-
14             1+1>d;++1){
15             if(da.front()==1)da.pop_front();
16             if(db.front()==1)db.pop_front();
17             if(da.size()&&db.size()){
18                 d=a[da.front()]+b[db.front()];
19             }
20             ans=max(ans,r-1+1);
21         }
22     }
23     printf("%d\n",ans);
24 }

```

12.3 最大矩形

```

1 LL max_rectangle(vector<int> s){
2     stack<pair<int,int>> st;
3     st.push(make_pair(-1,0));
4     s.push_back(0);
5     LL ans=0;
6     for(size_t i=0;i<s.size();++i){
7         int h=s[i];
8         pair<int,int> now=make_pair(h,i);
9         while(h<st.top().first){
10             now=st.top();
11             st.pop();
12             ans=max(ans,(LL)(i-now.second)*now.
13                 first);
14         }
15         if(h>st.top().first){

```

```

15     st.push(make_pair(h,now.second));
16 }
17 }
18 return ans;
19 }

```

13 other language

13.1 java

13.1.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10         IOException
11         Scanner sc = new Scanner(new FileReader(
12             "a.in"));
13         PrintWriter pw = new PrintWriter(new
14             FileWriter("a.out"));
15         int n,m;
16         n=sc.nextInt();//读入下一个INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #"+ci+": easy for
21                 output");
22         }
23
24         pw.close();//关闭流并释放，这个很重要，
25         否则是没有输出的
26         sc.close();//关闭流并释放
27     }
28 }

```

13.1.2 优先队列

```

1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10    });

```

13.1.3 Map

```

1 Map map = new HashMap();
2 map.put("sa","dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }

```

13.1.4 sort

```

1 static class cmp implements Comparator{
2     public int compare(Object o1,Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg,new cmp());
17 }

```

13.2 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

13.3 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

13.4 python output

```

1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"'.format('Jin', 'Kela'
10     ''))
11 # AAA Jin BBB "Kela!"
12
13 hello = 'hello, world\n'
14 hellos = repr(hello)
15 print(hellos) # 'hello, world\n'
16
17 x = 32.5
18 y = 40000
19 print(repr((x, y, ('spam', 'eggs'))))
20 # "(32.5, 40000, ('spam', 'eggs'))"
21
22 x = 7
23 print(eval('3 * x')) # 21

```

14 zformula

14.1 formula

14.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形，面積 = 內部格點數 + 邊上格點數/2-1

14.1.2 圖論

- 對於平面圖， $F = E - V + C + 1$ ， C 是連通分量數
- 對於平面圖， $E < 3V - 6$
- 對於連通圖 G ，最大獨立點集的大小設為 $I(G)$ ，最大匹配大小設為 $M(G)$ ，最小點覆蓋設為 $C_v(G)$ ，最小邊覆蓋設為 $C_e(G)$ 。對於任意連通圖：

$$(a) \quad I(G) + C_v(G) = |V|$$

$$(b) \quad M(G) + C_e(G) = |V|$$

- 對於連通二分圖：

$$(a) \quad I(G) = C_v(G)$$

$$(b) \quad M(G) = C_e(G)$$

- 最大權閉合圖：

$$(a) \quad C(u, v) = \infty, (u, v) \in E$$

$$(b) \quad C(S, v) = W_v, W_v > 0$$

$$(c) \quad C(v, T) = -W_v, W_v < 0$$

$$(d) \quad \text{ans} = \sum_{W_v > 0} W_v - \text{flow}(S, T)$$

- 最大密度子圖：

$$(a) \quad \text{求 } \max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

- $U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$
- $C(u, v) = W_{(u, v)}, (u, v) \in E$ 雙向邊
- $C(S, v) = U, v \in V$
- $D_u = \sum_{(u, v) \in E} W_{(u, v)}$
- $C(v, T) = U + 2g - D_v - 2W_v, v \in V$
- 二分搜 g ：
 $l = 0, r = U, \text{eps} = 1/n^2$
if $(U \times |V| - \text{flow}(S, T))/2 > 0$ $l = \text{mid}$
else $r = \text{mid}$
- $\text{ans} = \min_cut(S, T)$
- $|E| = 0$ 要特殊判斷

7. 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色，給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集：完美消除序列從前往後能選就選
- 最小團覆蓋：最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列：將區間按造又端點由小到大排序
- 區間圖染色：用線段樹做

14.1.3 dinic 特殊圖複雜度

- 單位流： $O \left(\min \left(V^{3/2}, E^{1/2} \right) E \right)$
- 二分圖： $O \left(V^{1/2} E \right)$

14.1.4 0-1 分數規劃

$$x_i \in \{0, 1\} \cdot x_i \text{ 可能會有其他限制} \cdot \text{求 } \max \left(\frac{\sum B_i x_i}{\sum C_i x_i} \right)$$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解， $f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態(通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i];//D(i,g)
16         找出一組合法x[i]使f(g)最大;
17         p=0,q=0;
18         for(i:所有元素)

```

```

19 |         if(x[i])p+=B[i],q+=C[i];
20 |         g=p/q;//更新解・注意q=0的情況
21 |     }while(abs(Ans-g)>EPS);
22 |     return Ans;
23 | }

```

14.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 $n \oplus (n >> 1)$
- $SG(A+B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

14.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n lcm(i, j) = n \sum_{d|n} d \times \phi(d)$

14.1.7 排組公式

- k 卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 \dots + x_n = k, num = C_k^{n+k-1}$
- Stirling number of 2^{n^d} , n 入分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
- Bell number, n 入分任意多組方法數目
 - $B_0 = 1$
 - $B_n = \sum_{i=0}^n S(n, i)$
 - $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
 - $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
 - $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
 - From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
- Derangement, 錯排, 沒有人在自己位置上
 - $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} \dots + (-1)^n \frac{1}{n!})$
 - $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
 - From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$
- Binomial Equality
 - $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$

- $\sum_k \binom{l}{m+k} \binom{s}{n-k} = \binom{l+s}{l-m+n}$
- $\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-n-m}$
- $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- $\binom{r}{m} \binom{m}{k} = \binom{r}{k} \binom{r-k}{m-k}$
- $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{n+1}$
- $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- $\sum_{k \leq m} \binom{m+r}{k} x^k y^{m-k} = \sum_{k \leq m} \binom{-r}{k} (-x)^k (x+y)^{m-k}$

14.1.8 幕次, 幕次和

- $a^{b \% p} = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
- $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
- $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
- $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
- $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
- $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
- $\sum_{j=0}^n C_j^{m+1} B_j = 0, B_0 = 1$
- 除了 $B_1 = -1/2$ · 剩下的奇數項都是 0
- $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

14.1.9 Burnside's lemma

- $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
- $X^g = t^{c(g)}$
- G 表示有幾種轉法 · X^g 表示在那種轉法下 · 有幾種是會保持對稱的 · t 是顏色數 · c(g) 是循環節不動的面數。
- 正立方體塗三顏色 · 轉 0 有 3^6 個元素不變 · 轉 90 有 6 種 · 每種有 3^3 不變 · 180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot \frac{1}{24} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) = \frac{57}{57}$

14.1.10 Count on a tree

- Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
- Unrooted tree:
 - Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$
 - Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
- Spanning Tree
 - 完全圖 $n^n - 2$

- 一般圖 (Kirchhoff's theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, ans = \det(A)$

15 Интернационал

15.1 ganadoQuote

```

1 | ¡Allí está!
2 | ¡Un forastero!
3 | ¡Agarrenlo!
4 | ¡Os voy a romper a pedazos!
5 | ¡Cógelo!
6 | ¡Te voy a hacer picadillo!
7 | ¡Te voy a matar!
8 | ¡Míralo, está herido!
9 | ¡Sos cerdo!
10 | ¿Dónde estás?
11 | ¡Detrás de tí, imbécil!
12 | ¡No dejes que se escape!
13 | ¡Basta, hijo de puta!
14 | Lord Saddler...
15 |
16 | ¡Mátalo!
17 | ¡Allí está!
18 | Morir es vivir.
19 | ¡Síííí, ¡Quiero matar!
20 | Muere, muere, muere....
21 | Cerebros, cerebros, cerebros...
22 | Cógedlo, cógedlo, cógedlo...
23 | Lord Saddler...
24 | Dieciséis.
25 |
26 | ¡Va por él!
27 | ¡Muérete!
28 | ¡Cógelo!
29 | ¡Te voy a matar!
30 | ¡Bloqueale el paso!
31 | ¡Te cogí!
32 | ¡No dejes que se escape!
33 |
34 | ¿Qué carajo estás haciendo aquí? ¡Lárgate, cabrón!
35 | Hay un rumor de que hay un extranjero entre nosotros.
36 | Nuestro jefe se encargará de la rata.
37 | Su "Las Plagas" es mucho mejor que la nuestra.
38 | Tienes razón, es un hombre.
39 | Usa los músculos.
40 | Se vuelve loco!
41 | ¡Hey, acá!
42 | ¡Por aquí!
43 | ¡El Gigante!
44 | ¡Del Lago!
45 | ¡Cógelo!
46 | ¡Cógenlo!
47 | ¡Allí!
48 | ¡Rápido!
49 | ¡Empieza a rezar!

```

```

50 | ¡Mátenlos!
51 | ¡Te voy a romper en pedazos!
52 | ¡la campana!
53 | Ya es hora de rezar.
54 | Tenemos que irnos.
55 | ¡Maldita sea, mierda!
56 | ¡Ya es hora de aplastar!
57 | ¡Mierda!
58 | ¡Puedes correr, pero no te puedes esconder!
59 | ¡Sos cerdo!
60 | ¡Está en la trampa!
61 | ¡Ah, que madre!
62 | ¡Vámonos!
63 | ¡Ándale!
64 | ¡Cabron!
65 | ¡Coño!
66 | ¡Agárrenlo!
67 | Cógerlo, Cógerlo...
68 | ¡Allí está, mátalos!
69 | ¡No dejas que se escape de la isla vivo!
70 | ¡Hasta luego!
71 | ¡Rápido, es un intruso!

```

15.2 Интернационал

```

1 | /*****
2 | L'Internationale,
3 |     Sera le genre humain.
4 |
5 |
6 |
7 |
8 |
9 |
10 |
11 |
12 |
13 |
14 |
15 | *****/
16 |
17 | Вставай, проклятьем заклеимённый,
18 | Весь мир голодных и рабов!
19 | Кипит наш разум возмущённый
20 | И в смертный бой вести готов.
21 | Весь мир насилья мы разрушим
22 | До основанья, а затем
23 | Мы наш, мы новый мир построим, –
24 | Кто был ничем, тот станет всем.
25 |
26 | Chorus
27 | Это есть наш последний
28 | И решительный бой;
29 | С Интернационалом
30 | Воспримет род людской!
31 |
32 | Никто не даст нам избавленья:
33 | Ни бог, ни царь и не герой!
34 | Добьёмся мы освобожденья
35 | Своею собственной рукой.
36 | Чтоб свергнуть гнёт рукой умелой,
37 | Отвоевать своё добро, –
38 | Вдувайте горн и куйте смело,

```

39 Пока железо горячо!

40

41 Chorus

42

43 Довольно кровь сосать, вампиры,
44 Тюрьмой, налогом, нищетой!
45 У вас — вся власть, все блага мира,
46 А наше право — звук пустой!
47 Мы жизнь построим по-иному —
48 И вот наш лозунг боевой:
49 Вся власть народу трудовому!
50 А дармоедов всех долой!

52 Chorus
53
54 Презренны вы в своём богатстве,
55 Угля и стали короли!
56 Вы ваши троны, тунейдцы,
57 На наших спинах возвели.
58 Заводы, фабрики, палаты –
59 Всё нашим создано трудом.
60 Пора! Мы требуем возврата
61 Того, что взято грабежом.

63 Chorus
64
65 Довольно королям в угоду
66 Дурманить нас в чаду войны!
67 Война тиранам! Мир Народу!
68 Бастуйте, армии сыны!
69 Когда ж тираны нас заставят
70 В бою героически пасть за них –
71 Убийцы, в вас тогда направим
72 Мы жерла пушек боевых!

74 Chorus
75
76 Лишь мы, работники всемирной
77 Великой армии труда,
78 Владеть землёй имеем право,
79 Но паразиты — никогда!
80 И если гром великого грянет
81 Над сворой псов и палачей, —
82 Для нас всё так же солнце станет
83 Сиять огнём своих лучей.

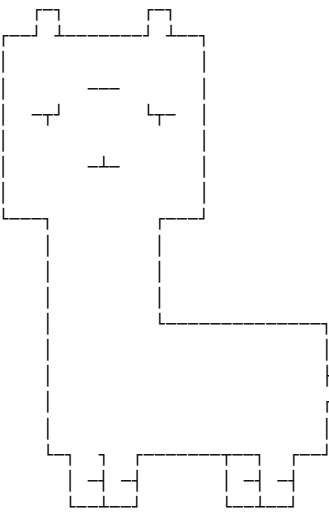
85 | Chorus

```

14 //
15 //      _oo0oo_
16 //      o8888888o
17 //      88" . "88
18 //      (| -_- |)
19 //      0\  =  /0
20 //
21 //      \   ___   /
22 //      .-; .-. ;.-/
23 //      |  ---  |
24 //      | _ \|
25 //      | |---' |
26 //      | | .- |
27 //      | |< <|> >|
28 //      | : :|
29 //      | _.. _..|
30 //      | | | |
31 //      | | | |
32 //      | | | |
33 //
34 // ~~~~~
35 //      佛祖保佑      永無BUG

```

54 #
55 #
56 #
57 #
58 #
59 #
60 #
61 #
62 #
63 #
64 #
65 #
66 #
67 #
68 #
69 #
70 #
71 #
72 #
73 #
74 #



神獸保佑 永無BUG!

[illegible]

15.3 保佑

- 1
- 2
- 3
- 4
- 5
- 6
- 7
- 8
- 9
- 10
- 11
- 12
- 13

ACM ICPC Team Reference - CCU-88boy

Contents

1 Computational Geometry	1	4.2 dijk	3	7.4 suffix-array	7	13 other language	10
1.1 convex-hull	1	4.3 Euler-Path	3	8 Tarjan	7	13.1 java	10
1.2 最近點對	1	4.4 SCC	3	8.1 dominator tree	7	13.1.1 文件操作	10
1.3 李超線段樹	1	5 Linear Programming	3	8.2 tnfsb017 2 sat	7	13.1.2 優先队列	10
2 Data Structure	1	5.1 simplex	3	8.3 橋連通分量	8	13.1.3 Map	10
2.1 persistent-segment-tree	1	6 Number Theory	3	8.4 雙連通分量 & 割點	8	13.1.4 sort	10
2.2 segment tree lazytag	1	6.1 basic	3	9 Tree Problem	8	13.2 python heap	10
2.3 treap	2	6.2 bit set	4	9.1 find-centroid	8	13.3 python input	10
3 Flow	2	6.3 cantor expansion	4	9.2 hld	8	13.4 python output	10
3.1 dinic	2	6.4 FFT	4	9.3 lca	8	14 zformula	10
3.2 Edmonds-Karp	2	6.5 find real root	4	10 compare	9	14.1 formula	10
4 Graph	2	6.6 FWT	4	10.1 check	9	14.1.1 Pick 公式	10
4.1 bellman-ford	2	6.7 LinearCongruence	5	10.2 gen	9	14.1.2 圖論	10
		6.8 Lucas	5	11 default	9	14.1.3 dinic 特殊圖複雜度	10
		6.9 Matrix	5	11.1 debug	9	14.1.4 0-1 分數規劃	10
		6.10 MillerRobin	5	11.2 ext	9	14.1.5 學長公式	11
		6.11 NTT	5	11.3 IncStack	9	14.1.6 基本數論	11
		6.12 Simpson	6	11.4 input	9	14.1.7 排組公式	11
		6.13 外星模運算	6	12 other	9	14.1.8 冪次, 冪次和	11
		6.14 數位統計	6	12.1 WhatDay	9	14.1.9 Burnside's lemma	11
		6.15 質因數分解	6	12.2 上下最大正方形	9	14.1.10 Count on a tree	11
		7 String	6	12.3 最大矩形	9	15 Интернационал	11
		7.1 ac 自動機	6			15.1 ganadoQuote	11
		7.2 KMP	6			15.2 Интернационал	11
		7.3 manacher	7			15.3 保佑	12

ACM ICPC Judge Test - CCU-88boy

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```